
University of
Nottingham
UK | CHINA | MALAYSIA
COMP-2024 / COMP-2039
Artificial Intelligence Methods
Lecture 2
Components of Heuristics Search Methods and Hill Climbing

Learning Outcomes

IDENTIFY

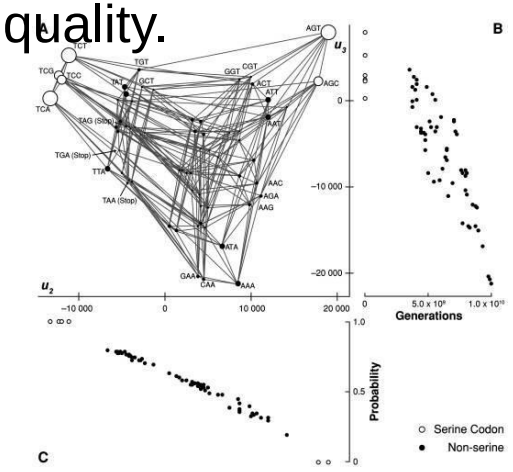
1. Main components of heuristic search/optimisation methods
2. Representation
3. Neighbourhoods
4. Evaluation Function
5. Hill climbing methods
6. Reading: Performance Analysis of Stochastic Local Search Methods – Preliminaries

University of Nottingham
UK | CHINA | MALAYSIA

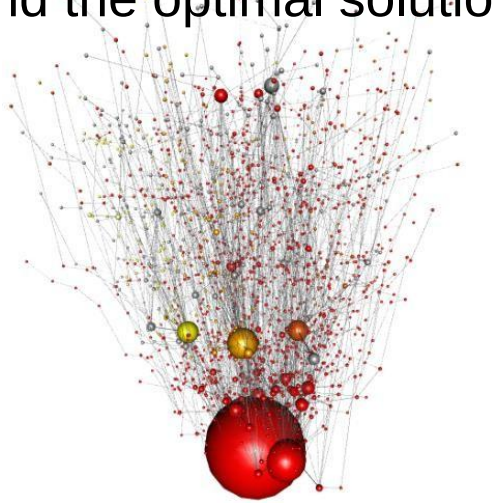
Main components of heuristic search/ optimisation methods

Problem: Search for an Optimal Solution

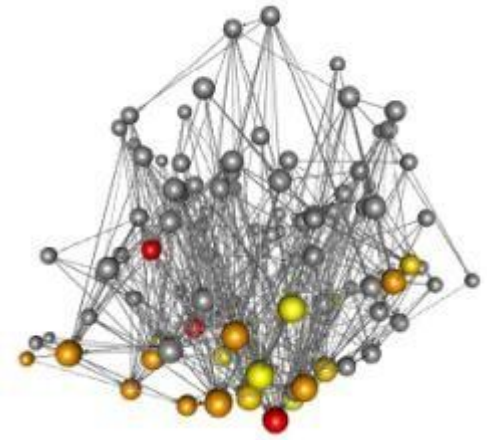
These examples represent different types of optimization problems, from discrete (RNA sequencing, TSP, flowshop scheduling) to continuous (function minimization).
The underlying search involves heuristic or exact methods to **navigate large, complex solution spaces efficiently**.
Goal: Use algorithms to find the optimal solution, balancing computational resources and solution quality.



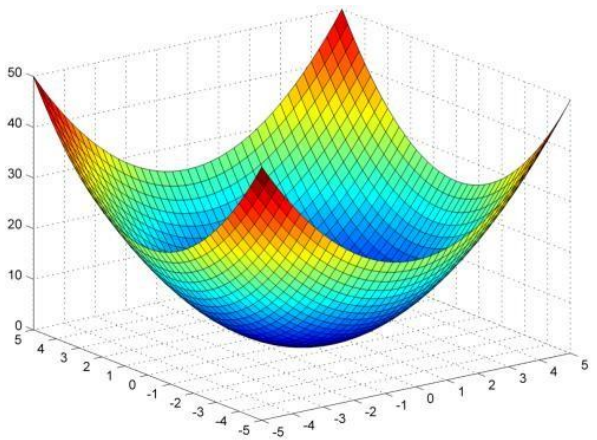
RNA Sequencing:
3 nucleotides



TSP:
pr1002



Permutation flowshop scheduling:
10 jobs and 7 machines



minimise
$f\ (x_1,x_2) = {x_1}^2+{x_2}^2$,
$-5.0 \leq x_1,x_2 \leq 5.0$

Main Components of Heuristic Search / Optimisation Methods

1. **Representation** (encoding) of candidate solutions
2. **Initialisation** (e.g., random)
3. **Evaluation function** (objective function)
4. **Neighbourhood relation** (move operators)
5. **Search process** (guideline)
6. Mechanism for **escaping** from local optima

Main Components of Heuristic Search / Optimisation Methods

1. **Representation** (encoding) of candidate solutions - defines how potential solutions are structured and encoded
2. **Initialisation** (e.g., random) - generates the starting set of candidate solutions, often using random values to promote diversity in the search space.
3. **Evaluation function** (objective function) - assesses the quality of candidate solutions by assigning a fitness score that guides the search process.

Main Components of Heuristic Search / Optimisation Methods

4. **Neighbourhood relation** (move operators) - specify how candidate solutions can be modified through move operators to explore the solution space.
5. **Search process** (guideline) - outlines the strategy for iteratively exploring and improving candidate solutions to find optimal results.
6. Mechanism for **escaping** from local optima - prevent the algorithm from getting stuck in local optima by encouraging exploration of the solution space.

University of Nottingham
UK | CHINA | MALAYSIA

Representation

1. Representation (Encoding) of Candidate Solutions

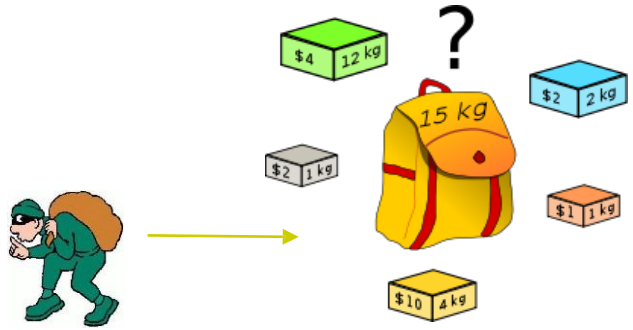
- Define how the problem state and the search space are represented or **encoded (in computers, mathematically)**
- Examples:
  - **Binary Strings**: For problems like the knapsack problem.
  - **Real Numbers**: For continuous optimization problems.
  - **Permutations**: For sequencing problems like the Traveling Salesman Problem (TSP).
- Ensures all encoded solutions represent valid candidates in the problem domain.
- A good representation simplifies the search and ensures feasibility of solutions.

Representation (Encoding of a Solution) – Characteristics

- Completeness: all solutions associated with the problem must be represented.
- Connexity: a *search path* must exist between any two solutions of the search space. Any solution of the search space, especially the global optimum solution, can be attained.
- Efficiency: the representation must be easy to manipulate by the search operators.

Representation - Binary Encoding

- **Binary encoding** is the most common
  - 10110010110010…1011



- E.g.: 0/1 Knapsack problem

  Fill the knapsack with as much value in goods as possible – which items to take?

  10011 ($8), 11110 ($15)

- Given a binary string of length N (representing N items), search space size is 2^N

Representation - Binary Encoding

- **Binary Encoding** for MAX-SAT Example:
  - C1: (x1 ∨ ¬x2 ∨ x3)
  - C2: (¬x1 ∨ x2)
  - C3: (x2 ∨ ¬x3)
- Candidate Solution Encoding
  - x1: 0 (False) or 1 (True)
  - x2: 0 (False) or 1 (True)
  - x3: 0 (False) or 1 (True)
  - Solution: x1 = True, x2 = False, x3 = True
  - Binary Encoding: 101

Representation - Permutation encoding

- E.g.: Travelling salesman problem, some sequencing problems.
- **Permutation encoding**
  - A candidate solution for 10 city

TSP instance would be:

1 5 3 2 10 4 7 9 8 6

- Given N cities (pubs), search space size is *N*!

A shortest-possible walking tour through the pubs of the UK (Nottingham)

Representation - Integer encoding

- **Integer encoding -** Integer encoding is a method of representing solutions to optimization problems using integers. Each integer represents a specific choice or option within the problem.
  - 1 3 4 5 5 5 4 1 1 … 2 2 1
- E.g.: Personnel rostering problem, timetabling problem, layout/structure optimization.

Representation – Integer Encoding

- **Scenario:** You have 5 different composite materials and need to choose one for each layer of a 15-layer composite structure to maximize sound absorption.
- **Encoding Choices:** Each layer can be represented by an integer from 1 to 5, where each integer corresponds to a specific material.
  - 1 3 4 5 5 5 4 1 1 … 2 2 1

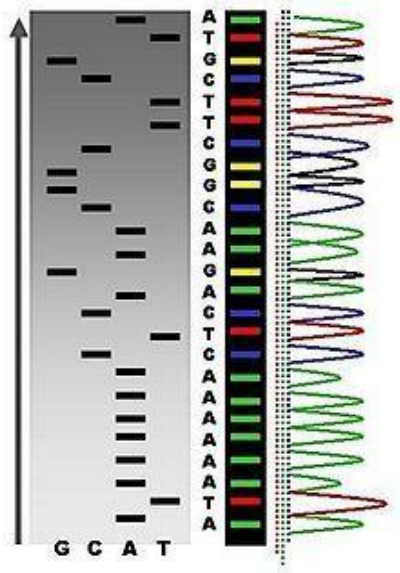
- Given an unlimited number of 5 different composite materials, which material would you use for each of the 15-layer composite structure maximising the sound absorption?
- For a general problem with M composite materials to form an N-layer composite structure, search space size (total number of possible combinations) is M^N.
- **Example:** If M=5 (materials) and N=15 (layers), the search space size would be 5^{15}

Representation - Value Encoding

- **Value Encoding -** represent solutions using real numbers or continuous values.
- Example: tuning hyperparameters in machine learning, physical measurements in engineering.
  - E.g.: Parameter/continuous optimization
    - 1.2324 5.3243 0.4556 2.3293 2.4545

Representation - Value Encoding

- E.g.: DNA sequencing is the process of determining the precise order of nucleotides within a DNA molecule. (Adenine, Guanine, Cytosine, and Thymine)
  - ATGCTTCGGCAAGACTCAAAAAATA

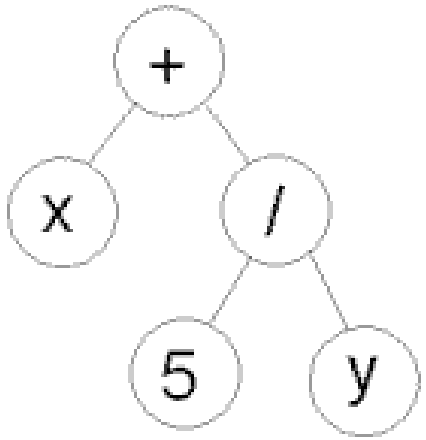


- E.g.: Planning
  - <(back), (back), (right), (forward), (left)>

Representation - Nonlinear Encoding

- **Nonlinear Encoding -** represent solutions using structures that can capture complex relationships, often through tree-like or hierarchical formats.
  - Tree Encoding – Genetic Programming
  - E.g.: Computers generating heuristics or heuristic components

(+ x (/ 5 y))



This tree structure can be translated into a mathematical expression:

- The expression represented by the tree is:

$$(x + \frac{y}{5})$$

Representation - Special Encodings

- Special Encodings
  - For example, **Random key encoding** is a technique where each candidate solution is represented by a set of random keys (values) that are typically generated in a continuous range (e.g., between 0 and 1). These keys are then used to determine the order or ranking of solutions rather than the values themselves.

1	2	3	4	5
0.2324	0.3243	0.4556	0.3293	0.4545
(1-0.2324	2-0.3243	4-0.3293	5-0.4545	3-0.4556) <= sorted

translates to

1 2 4 5 3

Maximum Satisfiability Problem (MAX-SAT)

The **Maximum Satisfiability Problem (MAX-SAT)** is an optimization problem in the realm of Boolean satisfiability. It seeks to determine the maximum number of clauses in a given Boolean formula that can be satisfied by an assignment of truth values to the variables. Here's a detailed explanation:

Key Concepts

1. **Boolean Formula**: MAX-SAT is typically defined over a formula in **Conjunctive Normal Form (CNF)**, which consists of a conjunction (AND) of clauses. Each clause is a disjunction (OR) of literals, where a literal is either a variable or its negation.
2. **Satisfaction**: A solution (assignment of truth values) is said to satisfy a clause if at least one of the literals in the clause evaluates to True. The goal of MAX-SAT is to maximize the number of satisfied clauses.

Example: Maximum Satisfiability Problem (MAX-SAT) Real-world Applications

probabilistic inference	[Park, 2002]
design debugging	[Chen, Safarpour, Veneris, and Marques-Silva, 2009]
	[Chen, Safarpour, Marques-Silva, and Veneris, 2010]
maximum quartet consistency	[Morgado and Marques-Silva, 2010]
software package management	[Argelich, Berre, Lynce, Marques-Silva, and Rapicault, 2010]
	[Ignatiev, Janota, and Marques-Silva, 2014]
Max-Clique	[Li and Quan, 2010; Fang, Li, Qiao, Feng, and Xu, 2014; Li, Jiang, and Xu, 2015]
fault localization	[Zhu, Weissenbacher, and Malik, 2011; Jose and Majumdar, 2011]
restoring CSP consistency	[Lynce and Marques-Silva, 2011]
reasoning over bionetworks	[Guerra and Lynce, 2012]
MCS enumeration	[Morgado, Liffiton, and Marques-Silva, 2012]
heuristics for cost-optimal planning	[Zhang and Bacchus, 2012]
optimal covering arrays	[Ansótegui, Izquierdo, Manyà, and Torres-Jiménez, 2013b]
correlation clustering	[Berg and Järvisalo, 2013; Berg and Järvisalo, 2016]
treewidth computation	[Berg and Järvisalo, 2014]
Bayesian network structure learning	[Berg, Järvisalo, and Malone, 2014]
causal discovery	[Hyttinen, Eberhardt, and Järvisalo, 2014]
visualization	[Bunte, Järvisalo, Berg, Myllymäki, Peltonen, and Kaski, 2014]
model-based diagnosis	[Marques-Silva, Janota, Ignatiev, and Morgado, 2015]
cutting planes for IPs	[Saikko, Malone, and Järvisalo, 2015]
argumentation dynamics	[Wallner, Niskanen, and Järvisalo, 2016]
...	

https://www.cs.helsinki.fi/group/coreo/aaai16-tutorial/aaai16-maxsat-tutorial.pdf

Example: Maximum Satisfiability Problem (MAX-SAT) – Binary Encoding

CNF Representation

Given the clauses:

1. $C_1 : (x_1 \vee \neg x_2)$
2. $C_2 : (x_2 \vee x_3)$
3. $C_3 : (\neg x_1 \vee x_3)$

The CNF for these clauses can be represented as:

$$F = (x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \vee x_3)$$

Consider the following CNF formula:

$$C_1 : (x_1 \vee \neg x_2)$$

$$C_2 : (x_2 \vee x_3)$$

$$C_3 : (\neg x_1 \vee x_3)$$

- Clauses: C_1, C_2, C_3
- Possible assignment: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$
- Satisfied clauses:
  - C_1: True (because x_1 is True)
  - C_2: True (because x_3 is True)
  - C_3: True (because x_3 is True)

Representation Example: MAX-SAT Problem

1. Binary Encoding

- **Description**: Each variable in the Boolean formula is represented by a binary value (0 or 1).
- **Example**: For a formula with variables x_1, x_2, x_3:
  - Candidate solution: $[1, 0, 1]$
  - Interpretation: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$

2. Bit Vector

- **Description**: A compact representation where each bit corresponds to a variable's assignment.
- **Example**: For variables x_1, x_2, x_3, x_4:
  - Candidate solution: `1100` (binary representation)
  - Interpretation: $x_1 = \text{True}, x_2 = \text{True}, x_3 = \text{False}, x_4 = \text{False}$

3. Integer Representation

- **Description**: Use integers to represent variable assignments, where 0 denotes False and 1 denotes True.
- **Example**: For x_1, x_2, x_3:
  - Candidate solution: $[1, 0, 1]$
  - Interpretation: Similar to binary encoding.

Exercise – Maximum Satisfiability Problem

- MAX-SAT: Given a Boolean formula in conjunctive normal form – a conjunction ($\wedge$) of clauses, where a clause is a disjunction ($\vee$) of literals (e.g., x_0, x_1),
  - find the <u>maximum</u> number of clauses that can be satisfied by some truth assignment.

    E.g., $(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$ => Problem instance

- How would you represent a candidate solution (suggest an encoding)?

MAX-SAT Problem – Candidate solution representation

$x_0\ x_1$		number of clauses satisfied
0 0:	$(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$ is true	2
0 1:	$(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$ is true	2
1 0:	$(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$ is false	1
1 1:	$(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$ is false	1

maximum number of clauses satisfied is 2

MAX-SAT Problem – Search Space Size

- Another problem instance
  - $(x_0 \vee x_1) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee \neg x_1) \wedge (\neg x_0 \vee \neg x_1)$
  - The literals x_0 and x_1 can take values of True (1) or False (0). Each clause (in parentheses) is a disjunction (OR) of two literals. The conjunction (AND) of all clauses means that all clauses must be satisfied simultaneously for the formula to be satisfiable.
  - not satisfiable, however, 3 clauses can be satisfied by any assignment (multiple solutions to this MAX-SAT instance can be found) – contradiction.
- Given *N* Boolean literals, what is the number of possible configurations (search space size)?
  - 2^N

MAX-SAT Problem – Search Space Size

- $(x_0 \vee x_1) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee \neg x_1) \wedge (\neg x_0 \vee \neg x_1)$

- **not satisfiable**: the four clauses represent all possible truth assignments for x_0 and x_1 and are mutually exclusive. This means there is no combination of x_0 and x_1 that satisfies all four clauses at the same time.
- Any assignment of x_0 and x_1 satisfies 3 out of the 4 clauses, which is the maximum possible.
- Multiple solutions exist where 3 clauses are satisfied.

For example:

- If $x_0 = 1$ and $x_1 = 1$, the clause $(\neg x_0 \vee \neg x_1)$ is not satisfied.
- If $x_0 = 0$ and $x_1 = 0$, the clause $(x_0 \vee x_1)$ is not satisfied.

MAX-SAT Problem – Search Space Size

- $(x_0 \vee x_1) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee \neg x_1) \wedge (\neg x_0 \vee \neg x_1)$

- Given *N* Boolean literals, what is the number of possible configurations (search space size)?
  - 2^N

For this example with $N = 2$ (i.e., x_0 and x_1):

$$2^2 = 4 \text{ possible configurations: } (00, 01, 10, 11).$$

Representation Example: Traveling Salesman Problem – Permutation Encoding

Permutation Representation

A candidate solution is represented as a **permutation of the cities**:

Encoding:

- `[A, C, B, E, D]`

Interpretation:

- Start at city **A** → visit **C** → visit **B** → visit **E** → visit **D** → return to **A**.

Numerical Representation:

If the cities are indexed (e.g., A=1, B=2, C=3, D=4, E=5), the same route can be represented as:

- `[1, 3, 2, 5, 4]`

Representation Example: Traveling Salesman Problem

Distance Matrix (Optional)

If we have a distance matrix representing the distances between cities:

	A	B	C	D	E
A	0	10	15	20	25
B	10	0	35	25	15
C	15	35	0	30	20
D	20	25	30	0	10
E	25	15	20	10	0

Route Cost Calculation

For the permutation `[1, 3, 2, 5, 4]`:

1. Distance from A (1) → C (3): **15**
2. Distance from C (3) → B (2): **35**
3. Distance from B (2) → E (5): **15**
4. Distance from E (5) → D (4): **10**
5. Distance from D (4) → A (1): **20**

Total cost of the route:

15 + 35 + 15 + 10 + 20 = **95**

University of Nottingham
UK | CHINA | MALAYSIA

Initialisation

2. Initialization

- Generates an **initial solution** or **population of solutions** for the search.
- Can be **random** or guided by **problem-specific heuristics** for diversity and quality.

University of Nottingham
UK | CHINA | MALAYSIA

Evaluation/ Objective Function

3. Evaluation Function (Objective Function)

- Also referred to as **objective**, cost, fitness, penalty, etc.
  - Measures the *quality* or *fitness* of a solution, distinguishing between better and worse solutions.
- An **objective function (evaluation function)**
  - *guides the optimization process* by indicating how close a solution is to the optimal.
  - serves as a major link between the algorithm and the problem being solved.
  - provides an important *feedback* for the search process.
- Many types: (non) separable, uni/multi-modal, single/multi-objective, etc.

1. (Non) Separable Objective Functions

- **Separable Functions**:
  - The objective function can be expressed as a sum of functions, each depending on a single variable or a subset of variables.
  - Example: $f(x_1, x_2, \ldots, x_n) = f_1(x_1) + f_2(x_2) + \cdots + f_n(x_n)$.
  - These are computationally easier to optimize as variables can be optimized independently.
- **Non-Separable Functions**:
  - The variables are interdependent, and the objective function cannot be decomposed into separate components.
  - Example: $f(x_1, x_2) = x_1 x_2 + x_1^2$.
  - These are more challenging to optimize because changing one variable affects the entire solution.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

2. Uni-modal vs. Multi-modal Objective Functions

- **Uni-modal Functions**:
  - The function has a single optimal solution (global maximum or minimum).
  - Example: A convex function, such as $f(x) = x^2$, is uni-modal because it has one minimum.
- **Multi-modal Functions**:
  - The function has multiple local optima (maxima or minima), with one being the global optimum.
  - Example: $f(x) = \sin(x)$ in a bounded domain has multiple peaks and valleys.
  - These are more challenging for optimization algorithms, as they can get trapped in local optima.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

3. Single-Objective vs. Multi-Objective Functions

- **Single-Objective Functions**:
  - The optimization problem involves only one objective to maximize or minimize.
  - Example: Minimize $f(x) = x^2 + 3x + 5$.
- **Multi-Objective Functions**:
  - There are multiple conflicting objectives to optimize simultaneously.
  - Example: Minimize $f_1(x) = x^2$ and maximize $f_2(x) = \sin(x)$.
  - Solutions are often represented as a **Pareto Front**, where no single solution is better across all objectives.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

4. Other Related Terms

- **Convex vs. Non-Convex**:
  - **Convex** functions have a single global optimum and are easier to optimize.
  - **Non-convex** functions have multiple local optima, increasing difficulty.
- **Continuous vs. Discrete**:
  - Continuous objective functions have variables that can take any value within a range.
  - Discrete functions involve variables restricted to specific values (e.g., integers, categories).
- **Deterministic vs. Stochastic**:
  - **Deterministic** functions yield the same output for a given input every time.
  - **Stochastic** functions have some level of randomness or noise, making the output vary for the same input.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul & Dr Chen ZhiYuan, UNM

Evaluation Function

- Can include *penalties* for constraint violations (if not handled earlier).

- Constraints Handling
  - **Soft Constraints** (Penalty Methods): Handle constraints by adding penalties to the objective function.
  - **Infeasibility Tolerance**: Allow infeasible solutions but favor feasible ones during search.
  - **Repair Operators**: Modify infeasible solutions to make them feasible.

Evaluation Function

- Evaluation functions could be **computationally expensive.**

- **Exact** vs. **approximate**
  - **Exact Models**: These provide precise outputs based on the input data, but computing them can be resource-intensive.
  - **Approximate Models**: These are simpler representations that aim to mimic the behavior of exact models with reduced computational costs. They are often easier and faster to compute. (our main focus)

Evaluation Function Example: MAX-SAT

Problem Context:

In MAX-SAT, the goal is to maximize the number of satisfied clauses in a given Boolean formula in Conjunctive Normal Form (CNF).

Example

Consider a Boolean formula with 3 variables (x_1, x_2, x_3) and 4 clauses:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

Evaluation Function Example: MAX-SAT

1. Input: A candidate solution represented as a binary vector for the truth assignment of variables.

   For example:

   - Assignment: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$
   - Representation: `[1, 0, 1]`

2. Evaluation:

   - Evaluate each clause under the given assignment:
     - Clause 1 ($x_1 \vee \neg x_2$): **True**
     - Clause 2 ($\neg x_1 \vee x_3$): **True**
     - Clause 3 ($x_2 \vee \neg x_3$): **False**
     - Clause 4 ($x_1 \vee x_2 \vee x_3$): **True**

3. Output:

   - The evaluation function returns the total number of satisfied clauses: **3 (out of 4)**.

Evaluation Function Formula:

$$f(\text{assignment}) = \text{Number of satisfied clauses under the assignment}$$

MAX-SAT Problem – Evaluation function

- **Maximising: (our lab experiments will do this)**

  f_1= ***C*; (count the number of satisfied clauses)**

- **Minimising:**

  f_2 **= (No. of clauses –** ***C*); (No. of unsatisfied clauses)**

  f_3 **=** f_2 **/ (No. of clauses);**

- Example: $(\neg x_0 \vee x_1) \wedge (\neg x_0 \vee \neg x_1)$

  x_0 x_1

  - 1 1 f_1=1, f_2=1, f_3=0.5 change x_0 to 0
  - 0 1 f_1=2, f_2=0, f_3=0.0 (better solution)

TSP – Evaluation function

- TSP requires a search for a permutation

$\pi : \{0,\ldots, N\text{-}1\} \rightarrow \{0,\ldots, N\text{-}1\},$

using a cost matrix $C=[c_{ij}]$, where c_{ij} denotes the cost (assumed to be known) of the travel from city i to j, that minimizes the *path length*

$$f(\pi, C) = \sum_{i=0}^{N-1} c_{\pi(i),\pi((i+1) \bmod N)},$$

where $\pi(i)$ denotes the city at i-th location in the tour and

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

TSP – Evaluation function

Search for a Permutation

- **Permutation** π: The solution to the TSP is represented as a permutation of cities, denoted as $\pi : \{0, \ldots, N-1\}$. This means the order in which the cities are visited is crucial.
- **Cities**: Each city is indexed, and the salesman must determine the optimal order of visiting these cities.

Cost Matrix $C = [c_{ij}]$

- **Definition**: The cost matrix C contains the costs (distances, travel times, etc.) associated with traveling from city i to city j.
- **Cost Representation**: Each element c_{ij} in the matrix represents the cost of traveling directly from city i to city j. It's assumed that these costs are known.

TSP – Evaluation function

Objective Function

- **Minimizing Total Cost**: The goal is to minimize the total cost of the tour. The evaluation function is defined as:

$$f(\pi, C) = \sum_{i=0}^{N-1} c_{\pi(i),\pi((i+1) \mod N)}$$

  - Here, $\pi(i)$ denotes the city at the i-th position in the tour.
  - The summation calculates the total cost by adding up the costs of traveling from each city to the next in the order specified by the permutation π.
  - The use of $\mod N$ ensures that the last city connects back to the first city, completing the tour.

Example Cost Calculation

- **Distance Calculation**: The cost c_{ij} can be calculated using coordinates of the cities (if given), for example:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

  - Here, (x_i, y_i) and (x_j, y_j) represent the coordinates of cities i and j. The formula calculates the Euclidean distance between two points in a 2D space.

Evaluation Function for TSP

Problem Context:

In TSP, the goal is to minimize the total distance traveled while visiting all cities exactly once and returning to the starting city.

Example

Consider 4 cities with the following distance matrix:

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

A candidate solution is represented as a permutation of cities:

- Candidate Route: `[A, C, D, B, A]`

Evaluation Function for TSP - Example

1. Input: A candidate solution, e.g., `[A, C, D, B, A]`.
2. Evaluation:
   - Calculate the total distance of the route:
     - $A \rightarrow C = 15$
     - $C \rightarrow D = 30$
     - $D \rightarrow B = 25$
     - $B \rightarrow A = 10$
   - Total Distance: $15 + 30 + 25 + 10 = 80$
3. Output:
   - The evaluation function returns the total distance of the route: **80**.

Evaluation Function Formula:

$$f(\text{route}) = \sum_{i=1}^{n} \text{distance}(\text{city}_i, \text{city}_{i+1})$$

Where n is the total number of cities and city_{n+1} is the return to the starting city.

University of Nottingham
UK | CHINA | MALAYSIA

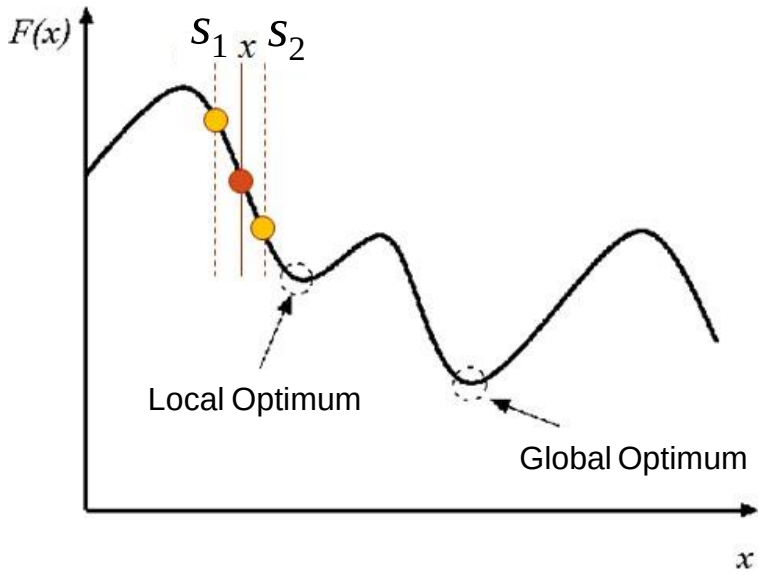
Neighbourhood Relation

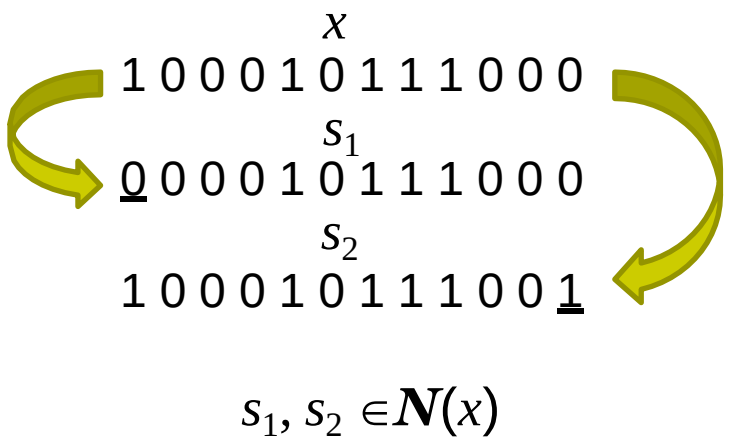
4. Neighborhood Relation (Move Operators)

- Defines the *relationship between solutions* (or states) in the search space.
- **Move operators** *generate new candidate solutions* by slightly modifying existing ones.
- Includes local adjustments (e.g., swaps, shifts, flips) or global changes (e.g., crossovers in Genetic Algorithms).

Neighbourhood

- A **neighbourhood** of a solution *x is a set of solutions* that can be reached from *x by a simple operator* (*move operator/heuristic*).





$s_1, s_2 \in \boldsymbol{N}(x)$

Example Neighbourhood for Binary Representation

- Bit-flip operator: flips a bit in a given solution.

- Hamming Distance between two-bit strings (vectors) of equal length is the number of positions at which the corresponding symbols differ. E.g., HD(011,010)=1, HD(0101,0010)=3 → distance between neighbours
- If the binary string is of size *n*, then the neighbourhood size is *n*.
- Example: **1** 0 1 0 0 0 1 1 → **0** 0 1 0 0 0 1 1

Neighbourhood size: 8, Hamming distance: 1

Example Neighbourhood for Integer/Value Representation

- **Random neighbourhood/move/perturbation/ assignment operator**: a discrete value is replaced by any other character of the alphabet.
- If the solution is of size *n* and alphabet is of size *k,* then the neighbourhood size is (k-1) n.

- Example: **5** 7 9 6 4 4 8 3 → **0** 7 9 6 4 4 8 3

Neighbourhood size: (10-1) × 8 = 72 (alphabet:0..9)

A D J E I F → **M** D J E I F

Neighbourhood size: (26-1) × 6 = 150 (alphabet:A..Z)

Example Neighbourhood for Permutation Representation I

- Adjacent pairwise interchange: swap adjacent entries in the permutation.
  - If permutation is of size *n*, then the neighbourhood size is *n*-1
  - Example: **5 1** 4 3 2 → **1 5** 4 3 2

- Insertion operator: take an entry in the permutation and insert it in another position.
  - Neighbourhood size: *n*(*n*-1)
  - Example: **5** 1 4 3 2 → 1 4 **5** 3 2

Example Neighbourhood for Permutation Representation II

- Exchange operator: arbitrarily selected two entries are swapped.
  - Example: **5** 4 3 **1** 2 → **1** 4 3 **5** 2

- Inversion operator: select two arbitrary entries and invert the sequence in between them.
  - Example: 1 **4 5 3** 2 → 1 **3 5 4** 2

Move Operator: TSP

2-opt Move:

In TSP, the goal is to find the shortest possible route that visits a set of cities and returns to the origin city. A typical move operator in TSP is the **2-opt move**, where two edges in the current tour are swapped to potentially shorten the total travel distance.

Example:

Consider a current route:

- City 1 → City 2 → City 3 → City 4 → City 5 → City 1

A 2-opt move would involve removing the edges between City 2 and City 3, and between City 4 and City 5, then reconnecting the two pairs of cities in the opposite order (i.e., City 2 → City 4 and City 3 → City 5). This can potentially reduce the overall distance traveled.

Move Operator: MAX-SAT

Flip a Variable:

In MAX-SAT, the goal is to maximize the number of satisfied clauses in a boolean formula. A simple move operator is to **flip the value of a randomly chosen variable**. This means if the variable is true, it becomes false, and if it is false, it becomes true. The effect of this flip is that it may increase or decrease the number of satisfied clauses in the formula, and the move operator aims to improve the solution iteratively.

Example:

Suppose the current assignment is:

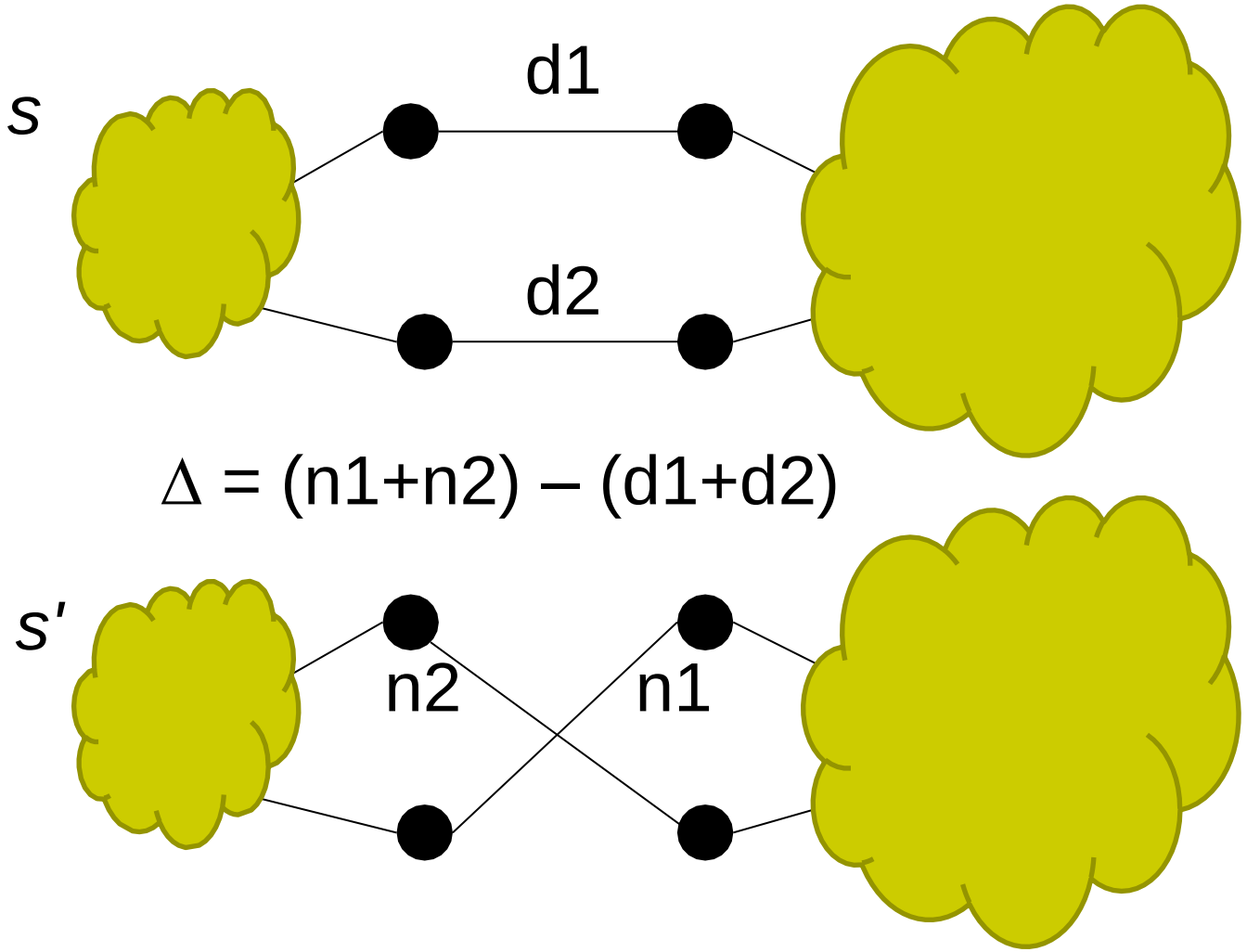
- $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$

A move could involve flipping x_2 from False to True. This might change the satisfaction of some clauses, but the objective is to improve the overall number of satisfied clauses.

Evaluation Function – Delta (Incremental) Evaluation

- Calculate **effects of differences** between current search position *s* and a neighbour *s′* on the evaluation function value.
- Evaluation function values often consist of independent contributions of solution components; hence, *f*(*s*′) can be efficiently calculated from *f*(*s*) by differences between *s* and *s*' in terms of solution components.
- This approach is crucial for the **efficient** implementation of heuristics and metaheuristics. It allows algorithms to explore the solution space more rapidly by evaluating only **small incremental changes** rather than recalculating the entire function, which saves computational resources.

Example: Delta Evaluation for TSP



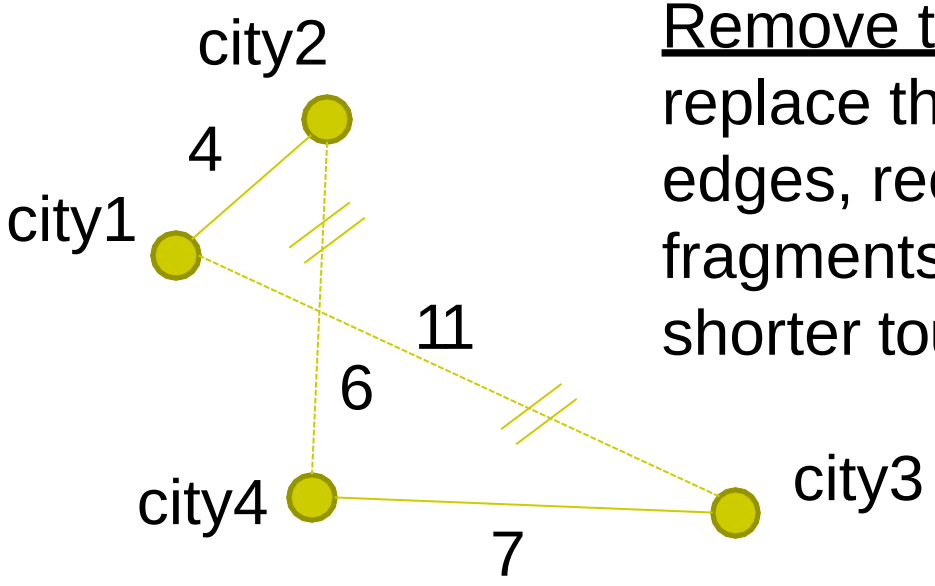
$f(s) = \Sigma$

$f(s') = \Sigma + \Delta$

Example: Delta Evaluation for TSP

$f(s) = \Sigma$

<city2, city1, city3, city4> : 28

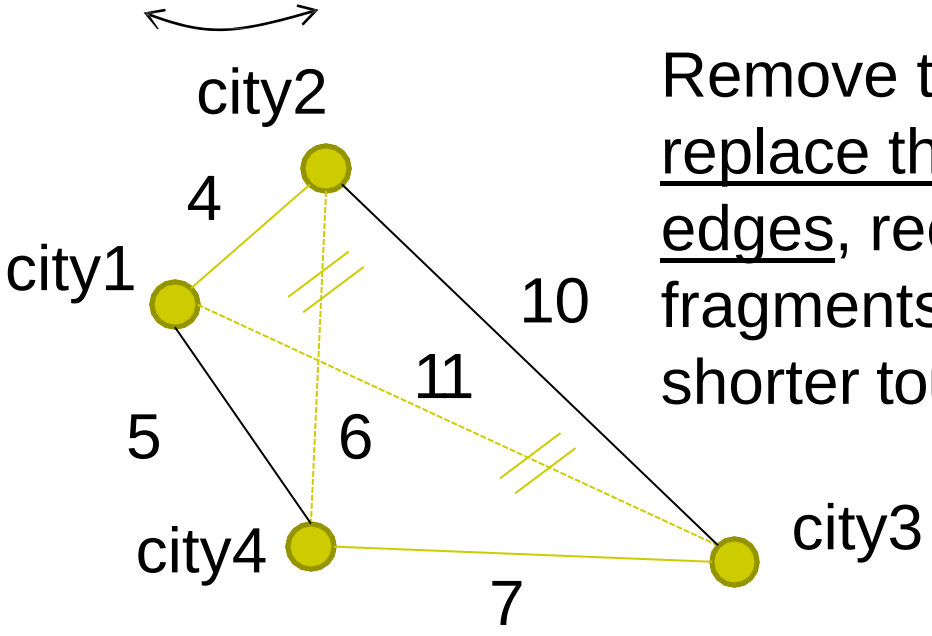


Remove two edges and replace them with two different edges, reconnecting the fragments into a new and shorter tour.

Example: Delta Evaluation for TSP

$f(s') = \Sigma + \Delta$

<city1, city2, city3, city4> : 26 (28+(-2))



Remove two edges and <u>replace them with two different edges</u>, reconnecting the fragments into a new and shorter tour.

$\Delta = (n1+n2) - (d1+d2)$

$\Delta = (5+10) - (11+6) = -2$

Example: Delta Evaluation for TSP

1. **Current Solution s**:
   - Represents a particular tour of the cities.
   - The total cost $f(s)$ is calculated as the sum of distances between the cities in the tour.

2. **Neighbor Solution s'**:
   - This is a modified tour derived from s by swapping two cities or making a small change.
   - The cost for s' is adjusted based on the changes made.

3. **Delta Calculation**:
   - The change in evaluation Δ is computed as:

$$\Delta = (n1 + n2) - (d1 + d2)$$

   Where:
   - $n1$ and $n2$ are the distances in the new solution s'.
   - $d1$ and $d2$ are the distances in the original solution s.

4. **Evaluation Function**:
   - The evaluation function for both solutions can be expressed as:

$$f(s) = \Sigma$$

   - The total cost is the summation of all relevant distances in the tour.

ACK: Prof. Ender Özcan, UNUK, Dr Tomas Maul

Summary

- Choosing an appropriate **encoding** to **represent** a candidate solution is crucial in heuristic optimization.
- **Initialisation** could influence the performance of an optimisation algorithm.
- **Evaluation function** guides the search process and fast evaluation is important.

University of Nottingham
UK | CHINA | MALAYSIA

Search Process (Guideline)

5. Search Process (Guideline)

- Refers to the **strategy** or **algorithm** or **heuristics** or **steps** used to explore the search space (e.g., hill climbing, simulated annealing, genetic algorithms) – more on metaheuristics
- Balances *exploration* (searching new areas) and *exploitation* (refining known good solutions) - e.g., annealing schedules, crossover vs. mutation rates – more on metaheuristics
- Utilize *parallel search* strategies to explore multiple regions of the search space simultaneously.
- **Stopping Criteria**: Use robust criteria such as stagnation detection, elapsed time, or a predefined solution quality.

University of Nottingham
UK | CHINA | MALAYSIA

Escaping from Local Optima

6. Mechanism for Escaping from Local Optima

- Prevents the search from getting *stuck in suboptimal solutions.*
- Examples:
  - **Diversity Preservation**: Techniques like maintaining diverse populations (in Genetic Algorithms) or periodic random restarts (in Hill Climbing).
  - **Adaptive Search Strategies**: Methods like Tabu Search, which remembers recently visited solutions to avoid cycling.
  - **Perturbation Techniques**: Introducing controlled randomness or large changes (e.g., simulated annealing cooling schedules or chaos-based perturbation).
  - **Hybridization**: Combine heuristic methods with exact methods (e.g., local search with Branch-and-Bound).

MAX-SAT – Escaping Local Optima

Mechanism: Tabu Search

Tabu Search is an optimization technique commonly used for escaping local optima in MAX-SAT. It prevents the algorithm from revisiting solutions that have been previously explored.

How It Works:

- **Initial Solution**: Begin with an initial random assignment of truth values to the variables in the MAX-SAT formula.
- **Neighborhood Search**: Generate a neighborhood of solutions by flipping the value of a variable (similar to the move operator discussed earlier).
- **Tabu List**: Maintain a list (called the "tabu list") of recently visited solutions or moves. The list keeps track of moves that have recently been made, and these moves are temporarily forbidden (i.e., marked as "tabu").
- **Aspiration Criteria**: Even if a move is tabu, it may be accepted if it results in a solution that improves the best-known solution. This is called an aspiration criterion.
- **Move Acceptance**: Select the best move from the neighborhood that is not tabu, or if all moves are tabu, select the best move that satisfies the aspiration criterion.
- **Termination**: The process continues for a set number of iterations or until a stopping condition is met (such as no improvement after a set number of iterations).

TSP – Escaping Local Optima

Mechanism: Simulated Annealing (SA)

Simulated Annealing is an effective approach to escape local optima in TSP. It works by occasionally accepting worse solutions (that increase the total tour length) to allow the algorithm to escape from local optima and explore the solution space.

How It Works:

- **Start with an initial tour**: Start with a random or greedy solution.
- **Perturbation**: Perform small perturbations (e.g., 2-opt or 3-opt moves) to create a new solution.
- **Acceptance criterion**: Evaluate the new solution and compare it with the current one. If the new solution is better, accept it. If the new solution is worse, accept it with a probability based on a temperature function that decreases over time (analogous to cooling in metallurgy). This probability allows the algorithm to occasionally accept worse solutions to escape local minima.
- **Cooling schedule**: The "temperature" is gradually decreased as the algorithm progresses, reducing the probability of accepting worse solutions, guiding the system toward convergence.
- **Termination**: The algorithm ends when the temperature is sufficiently low or a maximum number of iterations is reached.

Break

University of Nottingham
UK | CHINA | MALAYSIA

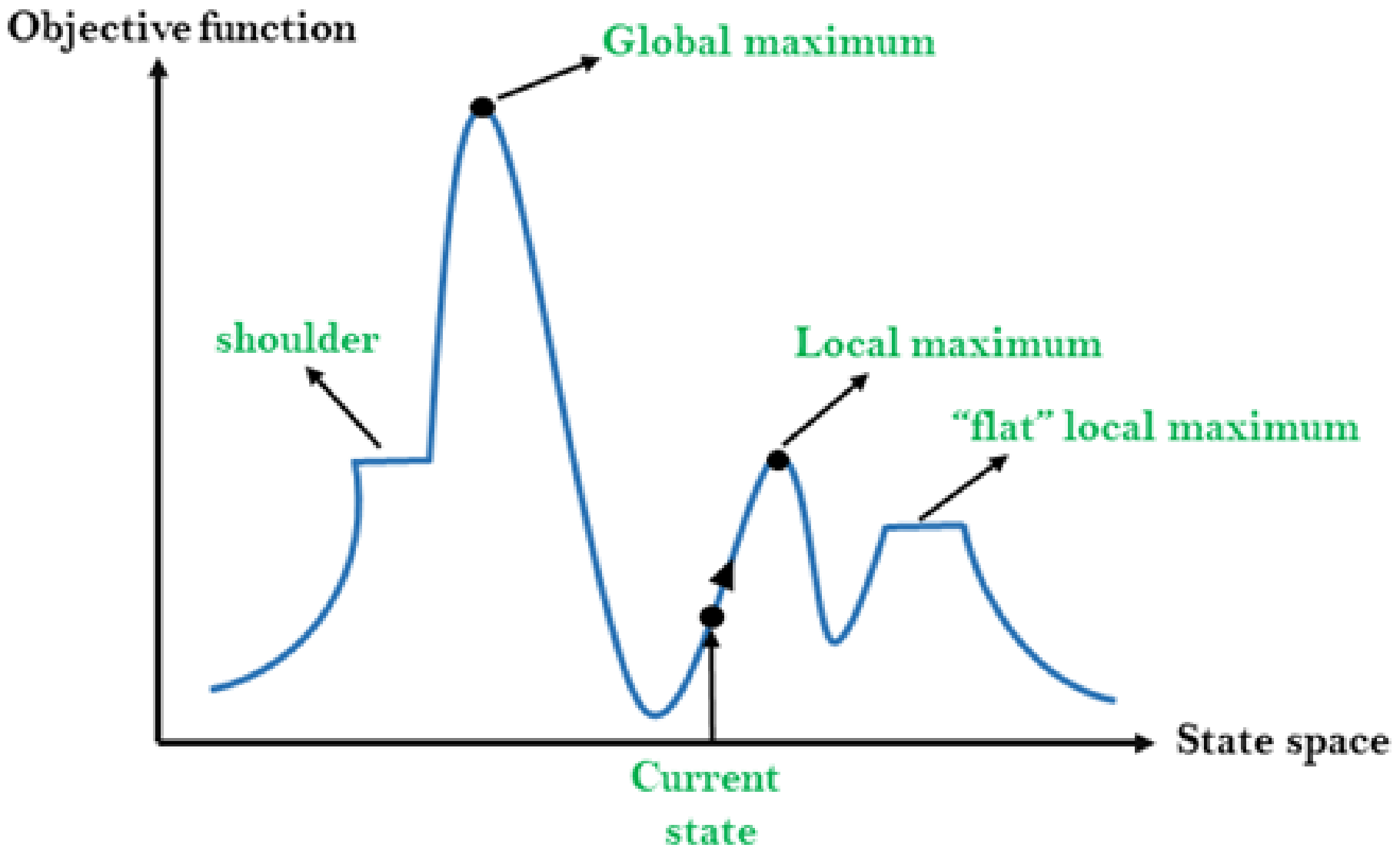
Hill Climbing Algorithms

Hill Climbing

- **Hill Climbing** is heuristic search algorithm used in mathematical optimization problems. It continuously moves towards the increasing value (or decreasing, depending on the problem) of the objective function to find the optimal solution.

- Applications:
  - Scheduling problems
  - Pathfinding (e.g., robotics, route optimization)
  - Tuning hyperparameters in machine learning models

Hill Climbing



Key Concepts

- **Objective Function**: The function to be maximized or minimized.
- **Local Optima**: A solution that is better than all neighbors but not necessarily the best globally.
- **Global Optima**: The best possible solution across the entire search space.
- **Greedy Approach**: Only considers immediate improvement.

Key Concepts

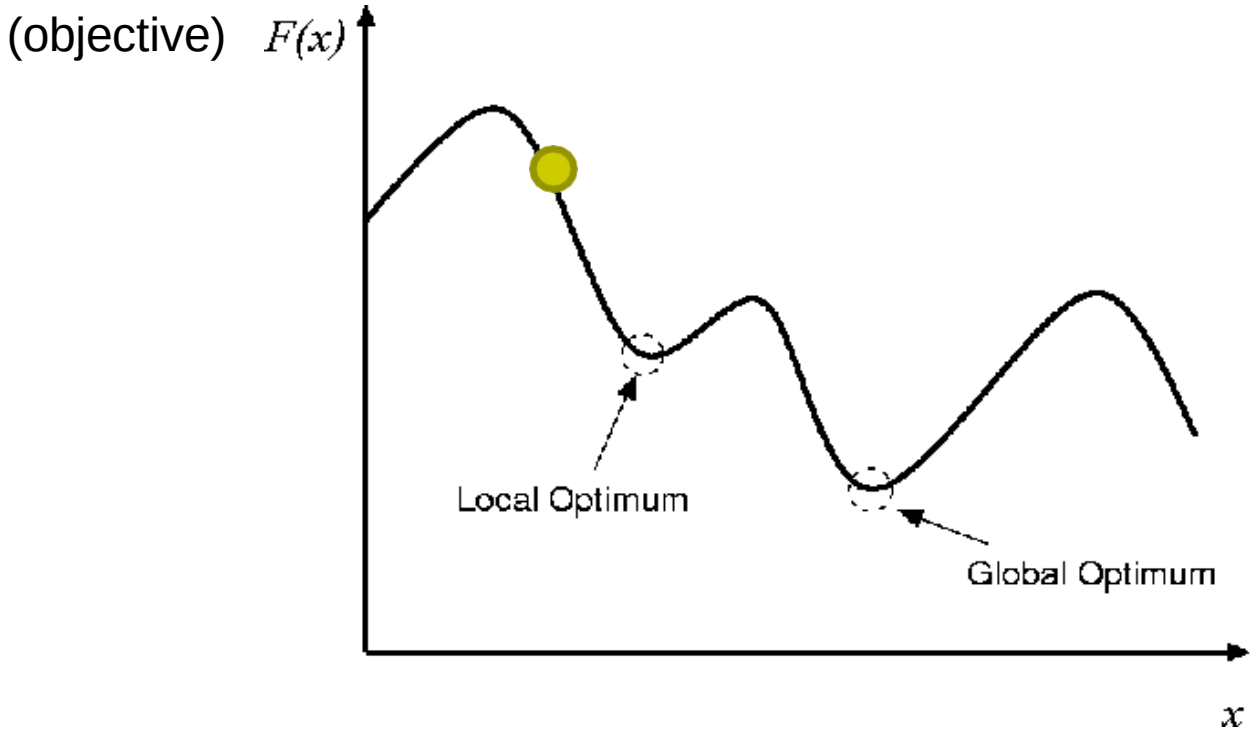
- **State Space**: All possible configurations of the problem.
- **Current State**: The current solution being evaluated.
- **Neighboring States**: Solutions that can be reached from the current state by making small changes (e.g., swapping two elements, changing a single variable).
- **Goal State**: The optimal solution.

Search Paradigms – Perturbative Heuristics/ Operators

- Mutational (diversification/exploration)

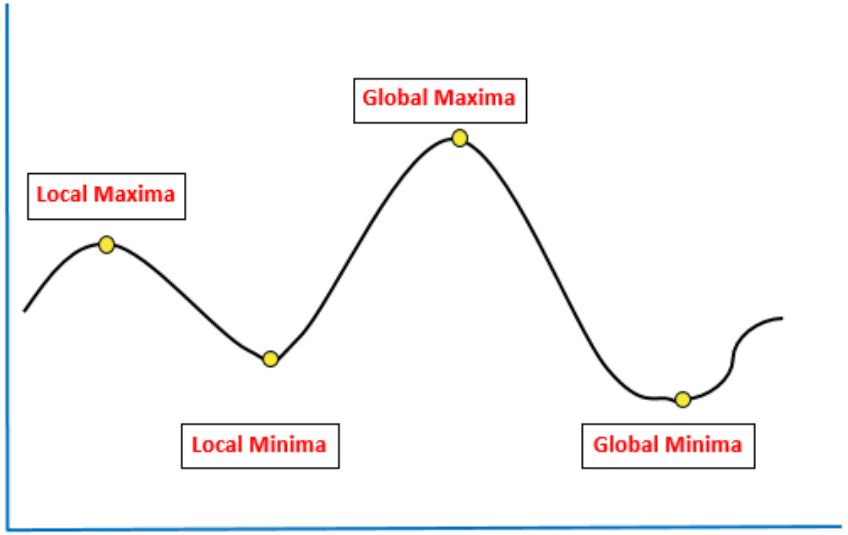
  vs.

- Hill-climbing (intensification/exploitation)



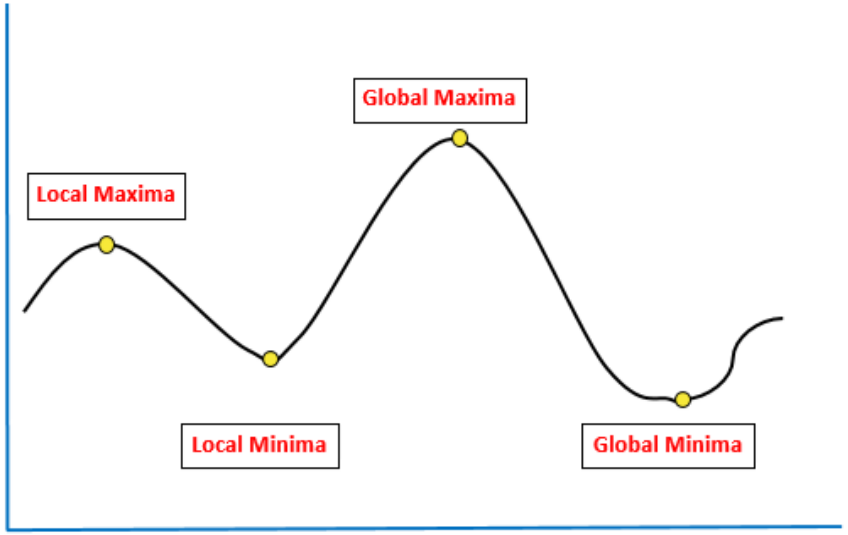
Hill Climbing Algorithm – Maximisation Problem

- A **local search algorithm** which constantly ***moves*** in the direction of increasing level/objective value (for a maximisation problem) to find the peak/the highest point of the landscape or best/near optimal solution to the problem.
- The hill climbing algorithm halts when it detects a peak value where no neighbour has a higher value.



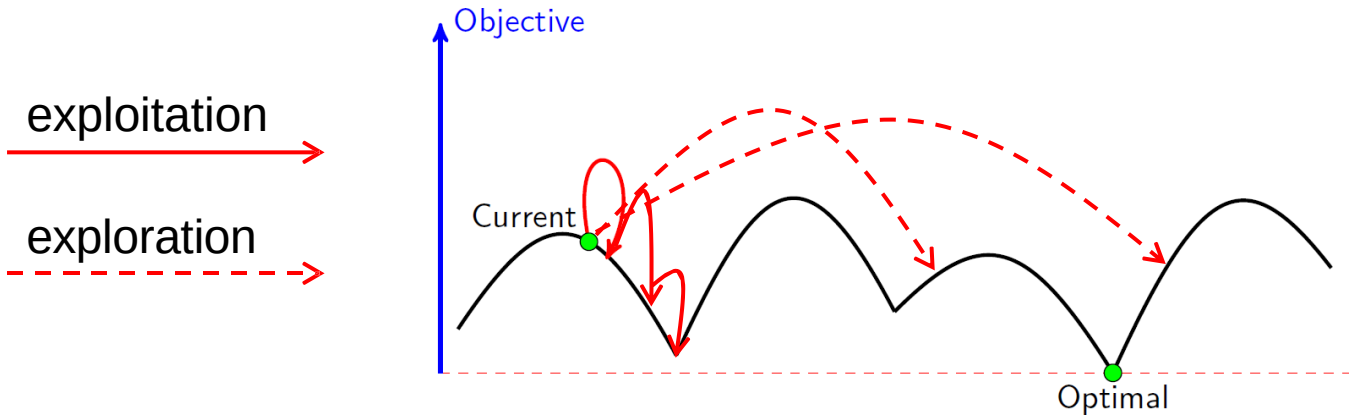
Hill Climbing Algorithm – Minimisation Problem

- A **local search algorithm** which constantly ***moves*** in the direction of decreasing level/objective value (for a minimisation problem) to find the *nadir/ the lowest point* of the landscape or best/near optimal solution to the problem.
- The hill climbing algorithm halts when it detects a nadir value where no neighbour has a lower value.



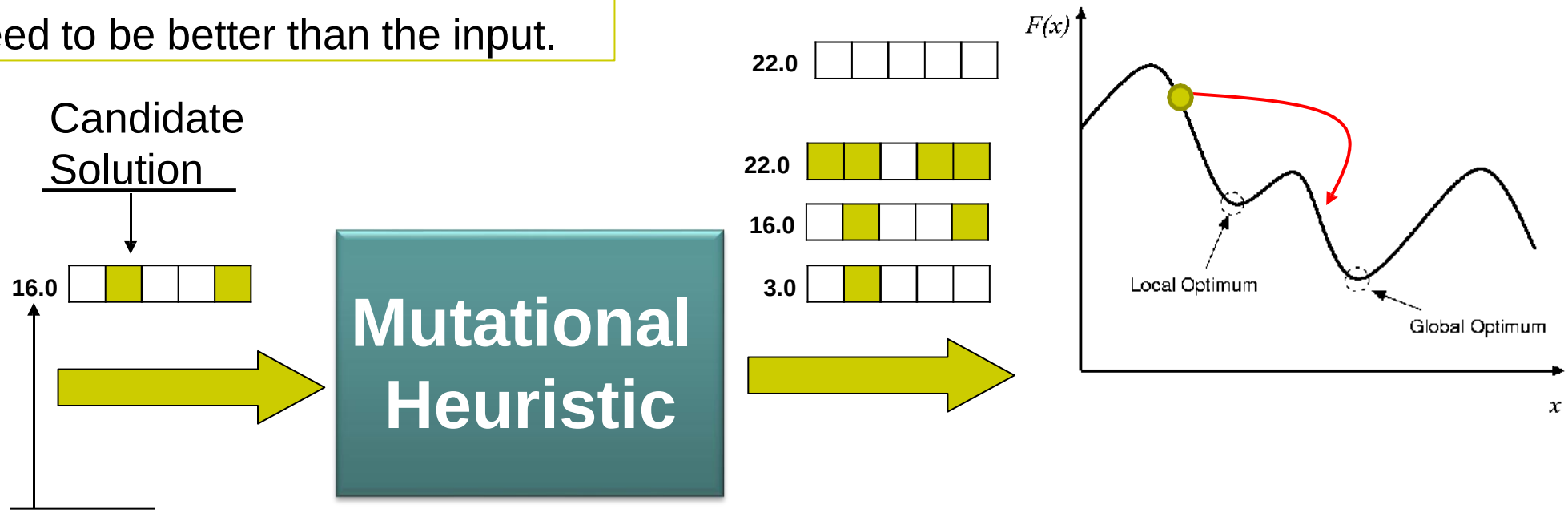
Hill Climbing versus Random Walk

- **A Hill-climbing** method ***exploits*** the best available solution for possible improvement but neglect exploring a large portion of the search space.
- **Random walk** (performs search in the search space, sampling new points with *equal probability*, e.g., *random bit flip, random swap*) ***explores*** the search space thoroughly but misses exploiting promising regions.



Mutational Heuristic/ Operator

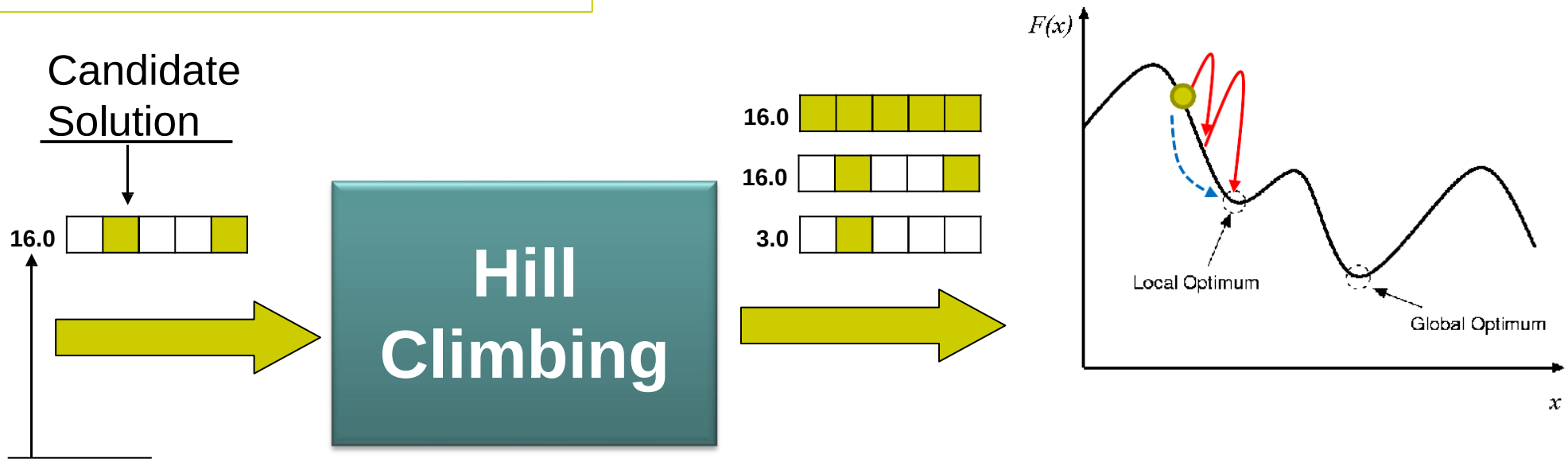
Processes a given candidate solution and generates a solution which is not guaranteed to be better than the input.



Minimising Fitness /Cost/Penalty/…
e.g., total number of constraint violations or a weighted sum of violations.

Hill Climbing Heuristic/ Operator

Processes a given candidate solution and generates a better or equal quality solution.



Minimising Fitness /Cost/Penalty/…
e.g., total number of constraint violations or a weighted sum of violations.

Mutational vs Hill-Climbing

Feature	Mutational (Exploration)	Hill-Climbing (Exploitation)
Focus	Broad exploration of the state space	Refining the current solution
Randomness	High	Low
Efficiency	Slower but thorough	Faster but narrower
Risk of Local Optima	Low	High
Typical Use	Early stages of search	Later stages or fine-tuning

Algorithm Steps

1. **Initialize**: Start with a random or specific initial solution.
2. **Evaluate**: Compute the objective (fitness) value of the current solution.
3. **Generate Neighbor**: Explore the neighboring solutions.
4. **Select Move**: Move to the neighbor with the highest improvement.
5. **Stop Condition**: Repeat steps 2–4 until no better neighbors exist or a maximum number of iterations is reached.

Example 1: Simple Numerical Optimization

Problem: Maximize $f(x) = -(x-3)^2 + 9$

This is a parabola with a maximum value of $f(x) = 9$ at $x = 3$.

Steps:

1. **Start**: Choose a random starting point, e.g., $x = 0$.
2. **Evaluate**: Compute $f(x) = -(0-3)^2 + 9 = 0$.
3. **Generate Neighbors**: Test small changes in x (e.g., $x = 1, x = -1$).
   - $f(1) = -(1-3)^2 + 9 = 4$
   - $f(-1) = -((-1)-3)^2 + 9 = -7$
4. **Move**: Select the neighbor with the highest $f(x)$, i.e., $x = 1$.
5. **Repeat**: Continue generating neighbors and moving:
   - $x = 2$: $f(2) = 8$
   - $x = 3$: $f(3) = 9$
6. **Stop**: No neighbors improve $f(x) = 9$.

Example 2: Travelling Salesperson Problem (TSP)

Problem: Minimize the distance traveled by visiting all cities once and returning to the starting city.

Cities: A, B, C, D

Distances:

From/To	A	B	C	D
A	0	4	1	3
B	4	0	2	5
C	1	2	0	6
D	3	5	6	0

Steps:

1. **Start**: Random tour $A \to B \to D \to C \to A$, total distance = $4 + 5 + 6 + 1 = 16$.
2. **Generate Neighbors**: Use a **2-opt move** (swap two edges).
   Example: Swap $B \to D$ with $B \to C$. New tour: $A \to B \to C \to D \to A$, total distance = $4 + 2 + 6 + 3 = 15$.
3. **Move**: Choose the neighbor with the shortest distance.
4. **Repeat**: Continue generating neighbors and improving.
5. **Stop**: When no swaps yield a shorter distance.

Pseudocode of a Generic Hill Climbing Algorithm

1. Pick an initial starting point (current state) in the search space.
2. Repeat
   i. Consider the neighbors of the current state.
   ii. Compare new point(s) in the *neighborhood* of the current state with the current state using an evaluation function and choose a new point with the best quality (among them) and move to that state.
3. Until there is no more improvement or when a predefined number of iterations is reached.
4. Return the current state as the solution state.

More on Hill Climbing

- Initial starting point(s) may be chosen,
  - randomly
  - use a constructive heuristic/operator(s)
  - according to some regular pattern
  - based on other information (e.g. results of a prior search)

- Variations of hill-climbing algorithms differ in the way a new solution/state/string is **selected** for comparisons with the current (incumbent) solution/state/string.

Variations of Hill-Climbing Algorithms

- **Simple Hill Climbing**: This version checks only one neighbor at a time. If the neighbor is better than the current solution, it moves to that neighbor.
- **Steepest-Ascent (or Descent) Hill Climbing**: Instead of checking one neighbor, this version evaluates all neighboring solutions and selects the one that offers the most significant improvement.
- **Random-Restart Hill Climbing**: To avoid local maxima, this variation repeatedly applies the hill climbing algorithm from different random starting points.
- **Stochastic Hill Climbing**: Instead of choosing the best neighbor, this version selects a neighbor at random and moves to it if it is better than the current solution. This introduces randomness to help explore the search space.

Types of Hill Climbing Heuristics

- **Simple Hill Climbing**: Evaluates one neighbor at a time and moves if the neighbor is better.
  - *Best improvement* (**steepest descent/ascent** - evaluate all neighbors and chooses the best one.)
  - *First improvement* (next descent/ascent)
- **Stochastic Hill Climbing** (randomly choose neighbours)
  - Random selection/random mutation hill climbing
- **Random-restart (shotgun)** hill climbing run the hill climbing algorithm multiple times from different random starting points in the solution space. This approach aims to mitigate the problem of getting stuck in local maxima, as different starting points can lead to exploring various areas of the search space.

How to define the size of the neighbourhood?

- **Simple Hill Climbing (first improvement)**: Typically considers the immediate neighbors of the current solution. For example, in a binary representation, flipping one bit creates a neighbor. Fixed neighborhood, e.g., all possible single-bit flips for a binary representation.
- **Steepest Ascent/Descent (best improvement)**: Evaluates all possible neighbors and selects the one with the best improvement (steepest increase or decrease in fitness).
- In algorithms like **Random Mutation Hill Climbing**, the neighborhood is not strictly defined, as random mutations can explore a broader solution space. This can help escape local optima.
- **Adaptive Neighborhood**: Adjust the neighborhood dynamically based on the search progress. For example, if many iterations yield no improvement, expand the neighborhood to include more distant solutions or allow for random mutations.

Best Improvement (steepest descent/ascent) Hill-climbing

```
bestEval = evaluate(currentSolution); improved = false;
for(j=0;(j<length[currentSolution]);j++){ // left to right scan, single pass
    bitFlip(currentSolution, j); // flips jth bit of current solution
    tmpEval = evaluate(currentSolution);
    if (tmpEval < bestEval) {   // strict improvement
        // remember the bit which yields the best value after evaluation
        bestIndex= j;
        bestEval = tmpEval;
        improved = true;
    } // end if
    bitFlip(currentSolution, j); // go back to the initial current solution
} // end for
if (improved)    bitFlip(currentSolution, bestIndex);
```

Exercise – Applying Best Improvement to a MAX-SAT Problem Instance

$$(\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$$

$x_0\ x_1\ x_2\ x_3 : f_2$

➡ 0 0 0 0:**1,** minimising f_2 (**No. of unsatisfied clauses**)

j

iteration#0: <u>1</u> 0 0 0:1

iteration#1: 0 <u>1</u> 0 0:1

iteration#2: 0 0 <u>1</u> 0:**0** ✓

iteration#3: 0 0 0 <u>1</u>:0

```
bestEval = evaluate(currentSolution); improved = false;
for(j=0;(j<length[currentSolution]);j++){ // left to right scan
  bitFlip(currentSolution, j); // flips jth bit of current solution
  tmpEval = evaluate(currentSolution);
  if (tmpEval < bestEval) {  // strict improvement
        // remember the bit which yields the best value after evaluation
        bestIndex= j;
        bestEval = tmpEval;
        improved = true;
  } // end if
  bitFlip(currentSolution, j); // go back to the initial current solution
} // end for
if (improved)  bitFlip(currentSolution, bestIndex);
```

Example: Best Improvement (Steepest Ascent) Hill-Climbing

1. **Evaluate Initial Solution**:
   - $x_1 = \text{False}, x_2 = \text{False}, x_3 = \text{True}$
   - Clauses satisfied:
     - $(x_1 \vee \neg x_2 \vee x_3) = (\text{False} \vee \text{True} \vee \text{True}) = \text{True}$
     - $(\neg x_1 \vee x_2) = (\text{True} \vee \text{False}) = \text{True}$
     - $(x_2 \vee \neg x_3) = (\text{False} \vee \text{False}) = \text{False}$
   - Total satisfied clauses = 2.
2. **Generate Neighbors**: Flip the value of each variable one at a time:
   - **Flip** x_1: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$
     - Clauses satisfied = 3 (all clauses satisfied).
   - **Flip** x_2: $x_1 = \text{False}, x_2 = \text{True}, x_3 = \text{True}$
     - Clauses satisfied = 3 (all clauses satisfied).
   - **Flip** x_3: $x_1 = \text{False}, x_2 = \text{False}, x_3 = \text{False}$
     - Clauses satisfied = 1.
3. **Select the Best Neighbor**:
   - Both flipping x_1 or x_2 results in 3 satisfied clauses, which is the maximum improvement.
   - Choose $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$ (arbitrarily if multiple best options exist).
4. **Repeat**: Since all clauses are satisfied, the algorithm terminates.

First Improvement (next descent/ascent) Hill-climbing

```
bestEval = evaluate(currentSolution);
for(j=0;(j<length[currentSolution]);j++){// single pass
   bitFlip(currentSolution, j);          // flips jth bit of solution producing s' from s
   tmpEval = evaluate(currentSolution);
   if (tmpEval < bestEval) // in case there is improvement
       bestEval = tmpEval;  // accept the bit flip
   else                                  // if no improvement, reject the bit flip
      bitFlip(currentSolution, j); // go back to s from s'
} // end for
```

Exercise – Applying First Improvement to a MAX-SAT Problem Instance

$$(\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$$

$x_0\ x_1\ x_2\ x_3 : f_2$

0 0 0 0:1, minimising f_2

1 0 0 0:1

0 0 0 0:1

0 1 0 0:1

0 0 0 0:1

✓ 0 0 1 0:0

0 0 1 1:0

```
bestEval = evaluate(currentSolution);
for(j=0;(j<length[currentSolution]);j++){// single pass
bitFlip(currentSolution, j);              // flips jth bit of solution producing s' from s
tmpEval = evaluate(currentSolution);
if (tmpEval < bestEval)                   // in case there is improvement
        bestEval = tmpEval;               // accept the bit flip
else                                      // if no improvement, reject the bit flip
        bitFlip(currentSolution, j);      // go back to s from s'
} // end for
```

Example: First Improvement Hill-Climbing

1. **Evaluate Initial Solution**:
   - $x_1 = \text{False}, x_2 = \text{False}, x_3 = \text{True}$
   - Total satisfied clauses = 2 (as evaluated earlier).
2. **Generate Neighbors and Stop at First Improvement**:
   - **Flip** x_1: $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$
     - Clauses satisfied = 3.
     - Since this is better than the current solution, move to this neighbor immediately without evaluating further neighbors.
3. **Repeat**:
   - Reevaluate from the new solution.
   - $x_1 = \text{True}, x_2 = \text{False}, x_3 = \text{True}$
   - All clauses are satisfied, so the algorithm terminates.

Best Improvement vs First Improvement

1. **Best Improvement**:
    - Evaluates **all neighbors** and chooses the best one.
    - In this example, evaluates x_1, x_2, x_3 flips before selecting the move.
    - Slower but ensures the steepest ascent in each step.
2. **First Improvement**:
    - Stops at the **first better solution** encountered.
    - In this example, stops after flipping x_1 without evaluating flips for x_2 or x_3.
    - Faster but may not always select the steepest ascent.

Davis's Bit Hill-climbing (DBHC)

- For problem binary encoded, e.g. MAX-SAT
- **Davis Bit Hill Climbing** focuses on manipulating individual bits in the binary representation of the solutions. When a bit is flipped (changed from 0 to 1 or vice versa), it generates a new candidate solution.
- The algorithm evaluates the new solution and compares it to the current one. If the new solution is better (i.e., has a higher fitness value), it replaces the current solution.

Davis's Bit Hill-climbing (DBHC)

```
bestEval = evaluate(currentSolution);
// creates a random permutation of integers (index values) from 0..length[currentSolution]-1
// in array perm, e.g., int perm[] ={0,1,2} (of length/size 3) becomes {1, 2, 0}
perm=createRandomPermutation(length[currentSolution]);
for(j=0;(j<length[currentSolution]);j++){        // single pass
   // flips the bit pointed by perm[j] of solution producing s' from s
   bitFlip(currentSolution, perm[j]);
   tmpEval = evaluate(currentSolution);
   if (tmpEval < bestEval)          // in case there is improvement
         bestEval = tmpEval;        // accept the bit flip
   else                             // if no improvement, reject the bit flip
         bitFlip(currentSolution, perm[j]); // go back to s from s'
} // end for
```

DBHC predetermines a random sequence to apply a hill climbing step and scans through the candidate solution according to it.

Exercise – Applying Davis's Bit Hill Climbing to a MAX-SAT Problem Instance

$$(\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$$

perm = [1, 2, 0, 3]

$x_0\ x_1\ x_2\ x_3 : f_2$

⇒ 0 0 0 0:1, minimising f_2

0 1 0 0:1 [**1**, 2, 0, 3]

0 0 0 0:1

✓ 0 0 1 0:0 [1, **2**, 0, 3] ← could stop here

1 0 1 0:0 [1, 2, **0**, 3]

0 0 1 0:0

0 0 1 1:0 [1, 2, 0, **3**]

Random Mutation Hill-climbing

```
bestEval = evaluate(currentSolution);
for(k=0;(k<noOfSteps);k++){ // single pass if k=length[solution]
   j=random(0, length[solution]);
   bitFlip(currentSolution, j); // flips jth bit of solution producing s' from s
   tmpEval = evaluate(currentSolution);
   if (tmpEval < bestEval)          // in case there is improvement
         bestEval = tmpEval;        // accept the bit flip
   else                             // if no improvement, reject the bit flip
         bitFlip(currentSolution, j); // go back to s from s'
} // end for
```

A bit is selected randomly and inverted for a number of iterations.

Exercise – Applying Random Mutation Hill Climbing to a MAX- SAT Problem Instance

$$(\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$$

random= [2, 0, 2, 3]

x_0 x_1 x_2 x_3: f_2

0 0 0 0:1, minimising f_2

0 0 1 0:0 [**2**, 0, 2, 3] ← could stop here

1 0 1 0:0 [2, **0**, 2, 3]

0 0 1 0:0

0 0 0 0:1 [2, 0, **2**, 3]

0 0 1 0:0

0 0 1 1:0 [2, 0, 2, **3**]

Hill Climbing Algorithm – Improving (< | >) vs. Non-worsening (≤ | ≥)

```
while (termination criteria not satisfied){
    …
    for …{ // single pass
            …
            if (tmpEval ≤ bestEval) {// accept non-worsening moves
                …
            }  // end if
    } // end for
            …
} //end while
```

Improving vs. Non-worsening

- **Improving Move**: If tmpEval is less than bestEval, the new solution is an improvement. The algorithm should accept this move as it leads to a *better solution*.

- **Non-Worsening Move**: If tmpEval is equal to bestEval, the new solution *does not worsen* the current best solution. The algorithm can also accept this move, allowing exploration of solutions that are as good as the best found so far.

Accepting Non-Worsening Moves

- The acceptance of **non-worsening moves** (i.e., solutions that do not make the situation worse) is significant for several reasons:
  - **Exploration**: It allows the algorithm to explore a broader search space, which may help avoid local optima. By accepting non-worsening solutions, the algorithm can potentially find better solutions later.
  - **Flexibility**: This approach provides flexibility in the search process, as it does not restrict the algorithm solely to improving moves.

Exercise – Applying Best Improvement to a MAX-SAT Problem Instance (Accepting Non-worsening Moves)

$$(\neg x_0 \vee x_1 \vee x_2) \wedge (x_0 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$$

$x_0\ x_1\ x_2\ x_3 : f_2$

➡ 0 0 0 0:1, minimising f_2 (No. of unsatisfied clauses)

j

iteration#0: <u>1</u> 0 0 0:1

iteration#1: 0 <u>1</u> 0 0:1

iteration#2: 0 0 <u>1</u> 0:0

iteration#3: 0 0 0 <u>1</u>:0 ✓

```
bestEval = evaluate(currentSolution); improved = false;
for(j=0;(j<length[currentSolution]);j++){ // left to right scan, single pass
   bitFlip(currentSolution, j); // flips jth bit of current solution
   tmpEval = evaluate(currentSolution);
   if (tmpEval ≤ bestEval) {  // accept non-worsening solutions
         // remember the bit which yields the best value after evaluation
         bestIndex= j;  bestEval =
         tmpEval;
         improved = true;
   } // end if
   bitFlip(currentSolution, j); // go back to the initial current solution
} // end for
if (improved)  bitFlip(currentSolution, bestIndex);
```

Hill Climbing Algorithms – Stopping Criteria: When to Stop

```
while (termination criteria not satisfied){
    …
    for …{ // single pass
            …

    } // end for
        …
} //end while
```

Hill Climbing Algorithms – When to Stop

- If the **target objective** is known (e.g., minimum value for f_2 is known which is 0), then the search can be stopped when that target objective value is achieved.
- Hill climbing could be applied repeatedly until a **termination criterion** is satisfied (e.g. maximum number of evaluations or iterations or generations (in evolutionary algorithms e.g. genetic) is exceeded which is a factor of the string length).
  - Note that there is no point applying Best Improvement, Next Improvement and Davis's (Bit) Hill Climbing if there is *no improvement after any single pass* over a solution.
  - Random Mutation Hill Climbing requires consideration (of stopping upon stopping condition) - to escape local optima and explore new areas

“No Improvement After a Single Pass”

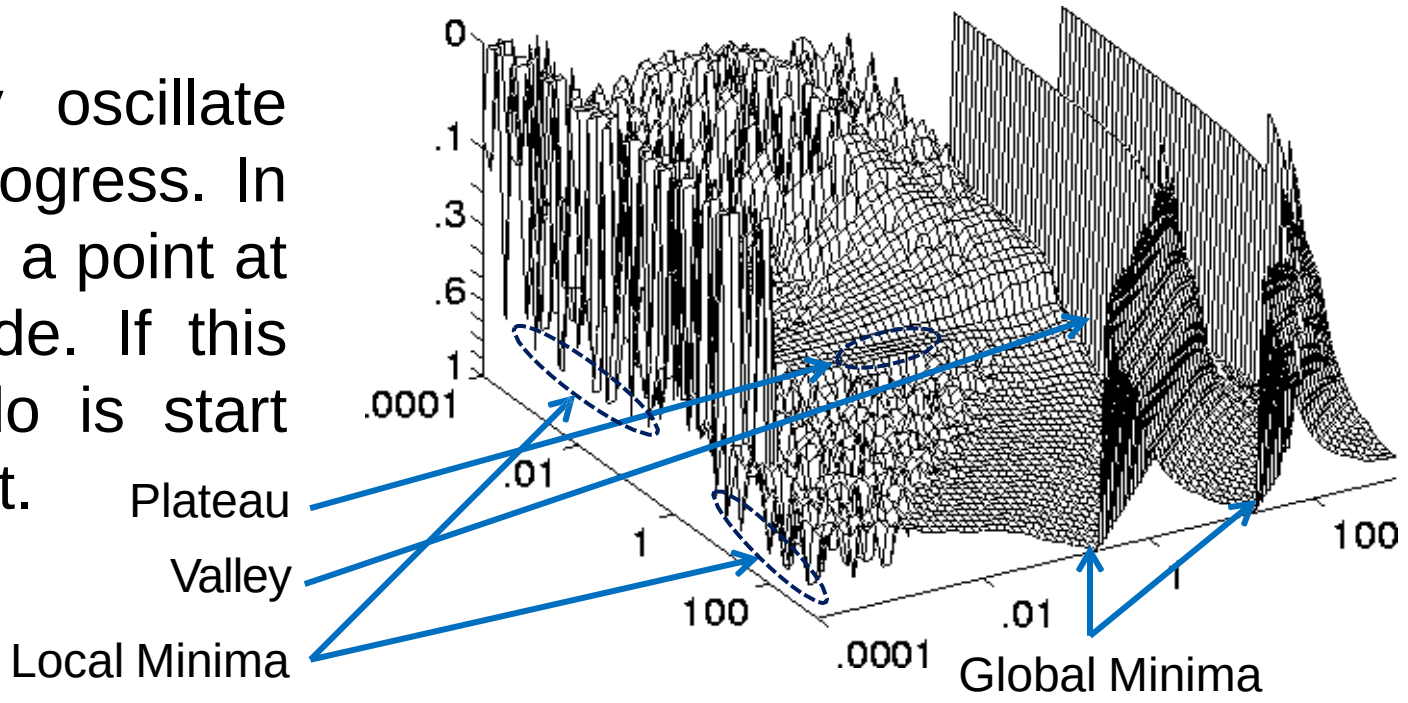
- If, after evaluating all possible neighboring solutions (or making a complete pass), none provide an improvement, it indicates that the algorithm is at a local optimum.
- At this point, continuing with these methods (Best Improvement, Next Improvement, Davis’s Hill Climbing) would be futile since they are designed to seek improvements. If none exist, the algorithm has effectively "stalled."

Strengths of Hill Climbing

- Very easy to implement, requiring:
  - a representation,
  - an evaluation function,
    - a measure that defines the neighbourhood around a point in the search space.

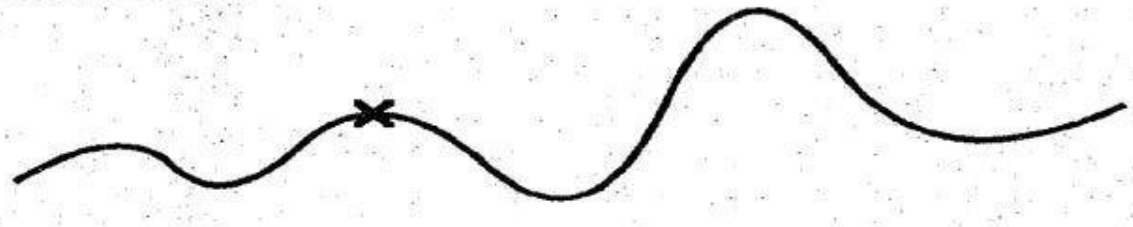
Weaknesses of Hill Climbing

- **Local Optimum:** If all neighboring states are worse or the same. The algorithm will halt even though the solution may be far from satisfactory.
- **Plateau (neutral space/shoulder):** All neighboring states are the same as the current state. In other words, the evaluation function is essentially flat. The search will conduct a random walk.
- **Ridge/valley:** The search may oscillate from side to side, making little progress. In each case, the algorithm reaches a point at which no progress is being made. If this happens, an obvious thing to do is start again from a different starting point.

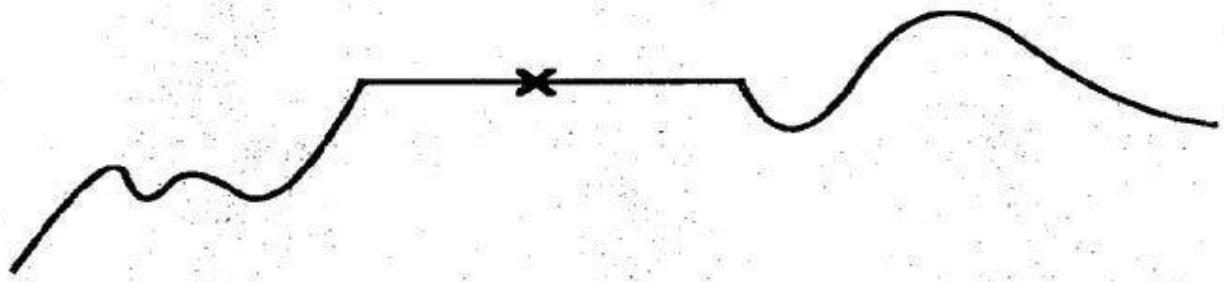


Hill-Climbing Dangers

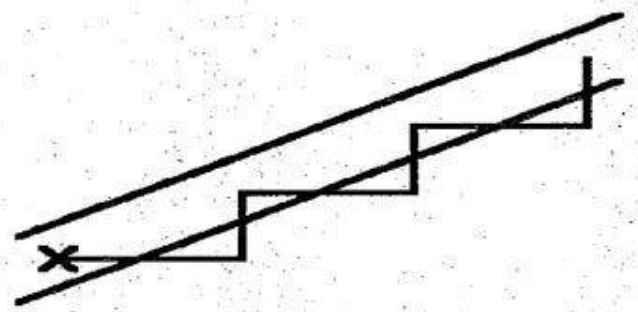
- Local maximum



- Plateau



- Ridge

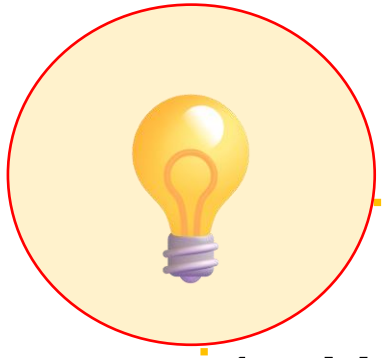


ACK: Prof. Ender Özcan, U

Weaknesses of Hill Climbing

- As a result, HC *may not find the optimal solution* and may *get stuck at a local optimum.*
- No information as to how much the discovered local optimum deviates from the global (or even other local optima).
- Usually, *no upper bound on computation time*.
- Success/failure of each iteration *depends on starting point* (initialisation)
  - success defined as returning a local or a global optimum

Summary



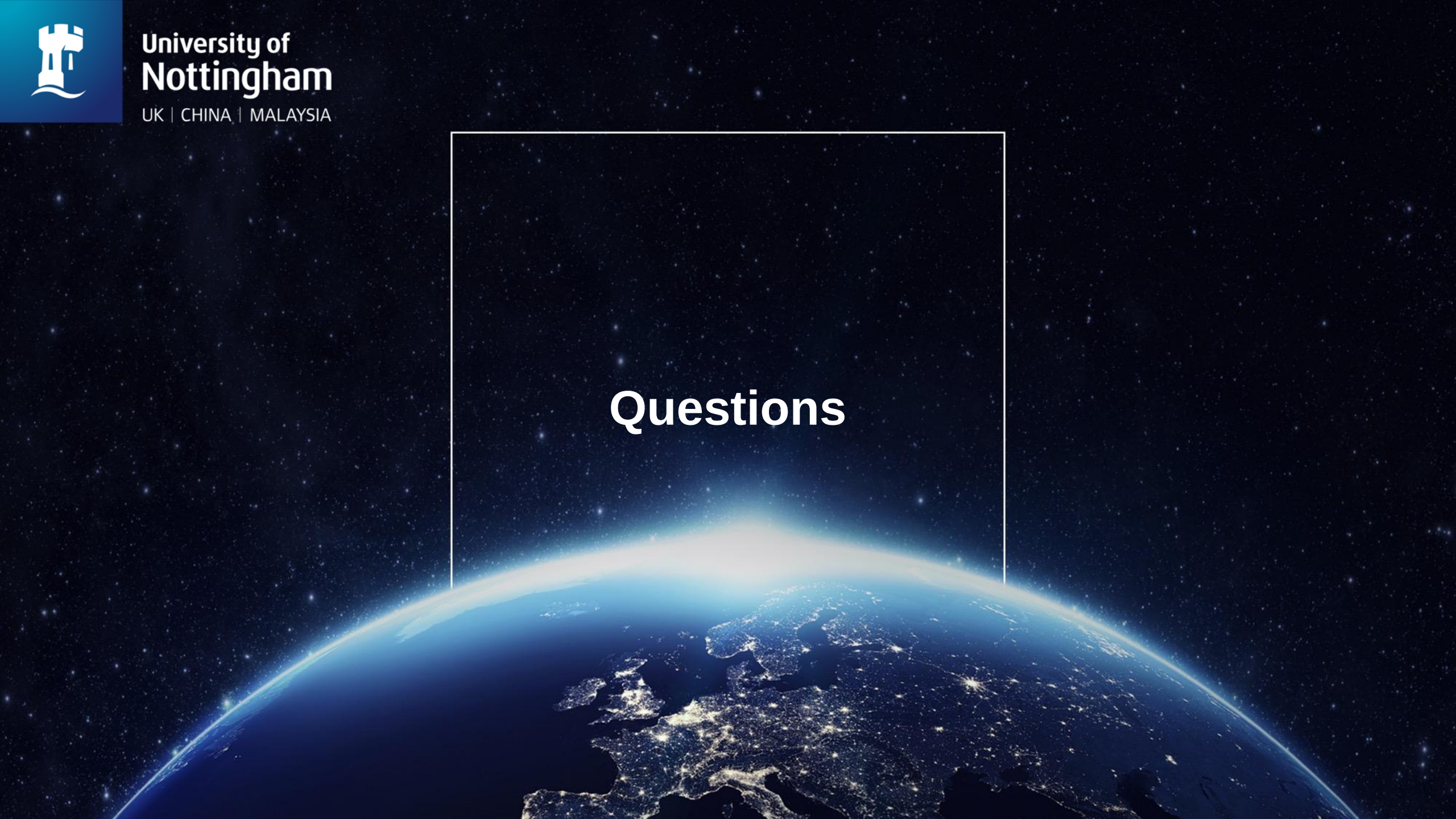
1. Main components of heuristic search/ optimisation methods
2. Representation
3. Neighbourhoods
4. Evaluation Function
5. Hill climbing methods
6. Reading: Performance Analysis of Stochastic Local Search Methods – Preliminaries

Home Exercise

- Assume that there are *N* examinations to timetable within *M* weeks (5 working/exam day per week), and 3 exam sessions per day. You are provided with a list of students and the exams that each one of them takes. The goal is to schedule all exams in such a way that there are no clashes for the students.
  - What representation would you use for the solution algorithm?
  - How would you design your objective function?
  - Is it possible to design delta evaluation?

University of Nottingham
UK | CHINA | MALAYSIA

Questions




University of
Nottingham
UK | CHINA | MALAYSIA
NEXT:
Metaheuristics